Topic: ScriptWell – Online Book Purchasing Store

PL/SQL PROJECT REPORT

Submitted by:

Student name

UID: 26MCA22333

Submitted to:

Mr. Ajay Yadav(E13382)

Submitted in partial fulfilment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

CHANDIGARH UNIVERSITY

University Institute of Computing

Punjab, India

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

BONAFIDE CERTIFICATE

This is to certify that the project report titled “ScriptWell – Online Book Purchasing Store” has been prepared as an academic project using Oracle Database Free 23ai and PL/SQL. The project demonstrates relational database design, stored procedures, stored functions, packages, triggers, explicit and implicit cursors, transaction control and structured exception handling. The database layer manages the book catalogue, user accounts, shopping carts, order processing, inventory auditing and administrative reporting, and it is accessed from a Java web application through JDBC CallableStatement.

This project report is submitted in partial fulfilment of the requirements for the degree of Master of Computer Applications (MCA).

Student Signature: _____ Project Supervisor: _____

Head of Department: _____ Date: _____

TABLE OF CONTENTS

Chapter 1: Introduction and Background Study .....PAGE 1 - 2

- 1.1 Project Overview
- 1.2 Problem Definition
- 1.3 Objectives and Scope
- 1.4 Technologies Used
- 1.5 Existing System and Proposed Solution

Chapter 2: System Requirements and Database Design .....PAGE 3 - 5

- 2.1 Hardware and Software Requirements
- 2.2 System Architecture
- 2.3 Data Flow and Transaction Workflow
- 2.4 Entity-Relationship Model and Data Dictionary
- 2.5 Functional and Non-Functional Requirements

Chapter 3: PL/SQL Implementation .....PAGE 6 - 9

- 3.1 Database Objects Overview
- 3.2 Packages and Stored Procedures
- 3.3 Stored Functions
- 3.4 Cursors and Transaction Control
- 3.5 Triggers
- 3.6 Exception Handling
- 3.7 Calling PL/SQL from Java (JDBC CallableStatement)

Chapter 4: Testing, Results and Conclusion .....PAGE 10 - 12

- 4.1 Testing Methodology
- 4.2 Test Cases and Validation
- 4.3 Results and Analysis
- 4.4 Limitations and Future Scope
- 4.5 Conclusion
- 4.6 Pushing the Project to GitHub
- References

CHAPTER 1:

INTRODUCTION AND BACKGROUND STUDY

1.1 Project Overview

ScriptWell is an online book purchasing store. Customers can browse 34 books in seven categories, search and filter the catalogue, keep a shopping cart, buy books with a simulated payment and track their orders. An administrator manages books, stock and orders from a dashboard. This report covers the **database layer** of the project, which is built entirely on **Oracle Database Free 23ai** and written in **PL/SQL**.

The main design decision is that the database owns the business rules. Stock checks, cart rules, price calculation, checkout, order cancellation, password hashing, validation and auditing all run as stored PL/SQL code. The Java web application never builds SQL strings. It only calls packaged procedures and functions through JDBC `CallableStatement`, and reads query results from `SYS_REFCURSOR` OUT parameters. As a result, every rule is applied the same way whichever client calls the database, and concurrent customers cannot leave the data in an inconsistent state.

1.2 Problem Definition

Many small e-commerce applications treat the database as passive storage and keep all logic in the application code. This causes several practical problems:

- **Overselling:** two customers who buy the last copy at the same moment can both succeed, because the stock check and the stock update are separate statements.
- **Partial data:** if an error occurs halfway through checkout, an order header can exist without its items, or stock can be reduced without an order.
- **Duplicated validation:** the same rules (price must be positive, quantity must be in range, e-mail must be unique) are re-implemented in several places and drift apart.
- **No audit trail:** nobody can later answer when stock changed, or who changed an order status.

The problem addressed by this project is to design a database that enforces these rules itself, using constraints, PL/SQL program units, triggers and transactions.

1.3 Objectives and Scope

Objectives

1. Design a normalised relational schema with primary keys, foreign keys, UNIQUE and CHECK constraints, indexes and a view.
2. Implement all core operations as **packages** (`pkg_user`, `pkg_book`, `pkg_cart`, `pkg_order`) with public and private sub-programs.
3. Write stand-alone **stored functions** and a **stored procedure** with OUT parameters for reusable calculations and dashboard statistics.
4. Use **explicit cursors**, cursor FOR loops and `SELECT ... FOR UPDATE` row locking to make checkout atomic and safe under concurrency.
5. Create **row-level triggers** for data cleaning and automatic auditing of stock and order status

changes.

6. Apply consistent **exception handling** with predefined, user-defined and application error codes (-20001 to -20010).

Scope

The database supports two roles, Customer and Administrator, and covers the full purchase cycle: registration and login, catalogue search, cart, checkout and Buy Now, order history and cancellation, and administration of books, inventory and orders. Payment is simulated, so no card data is stored. Multiple warehouses, coupons and real payment gateways are outside the scope.

1.4 Technologies Used

Table 1.1: Technologies and tools

Technology	Purpose in the project
Oracle Database Free 23ai	Relational database server (pluggable database FREEPDB1)
PL/SQL	Packages, procedures, functions, triggers, cursors and exception handling
SQL*Plus / SQL Developer	Running the install scripts, testing program units, inspecting results
Docker (gvenzl/oracle-free image)	Quick, repeatable local installation of Oracle Database Free
Java 17 + JDBC (ojdbc11)	Web application that calls PL/SQL through CallableStatement
Apache Tomcat 10.1	Runs the web front end used to demonstrate the database features
Git and GitHub	Version control and publishing of SQL scripts and source code

1.5 Existing System and Proposed Solution

Table 1.2: Comparison of the existing approach and ScriptWell

Aspect	Typical existing system	Proposed ScriptWell database
Business logic	Written in application code	Stored in PL/SQL packages inside Oracle
Stock control	Check then update, no locking	Row locking with FOR UPDATE inside one transaction
Failure handling	Partial rows may remain	Full ROLLBACK in exception handlers
Validation	Repeated in many places	Constraints, triggers and package validation in one place
Auditing	Manual or missing	Automatic logs written by triggers
Data access	Dynamic SQL strings	Only CallableStatement calls to stored programs

CHAPTER 2:

SYSTEM REQUIREMENTS AND DATABASE DESIGN

2.1 Hardware and Software Requirements

Table 2.1: Hardware and software requirements

Item	Requirement
Processor / Memory	64-bit dual-core CPU or better, minimum 8 GB RAM (Oracle Free uses about 2 GB)
Disk space	About 10 GB for Oracle Database Free and the project
Operating system	Windows 10/11, Linux or macOS (Docker or native Oracle installation)
Database	Oracle Database Free 23ai (also runs on Oracle 19c and later)
Database tools	SQL*Plus or Oracle SQL Developer
Application tier	JDK 17, Apache Tomcat 10.1, Eclipse IDE for Enterprise Java

2.2 System Architecture

ScriptWell has a three-tier architecture. The browser shows the pages, the Java application on Tomcat handles requests, and the Oracle database stores the data and runs the business logic. For this subject the important point is the boundary between the second and third tiers. Every DAO class in Java calls a PL/SQL program, and every program returns either scalar OUT parameters or a REF CURSOR (Figure 2.1).

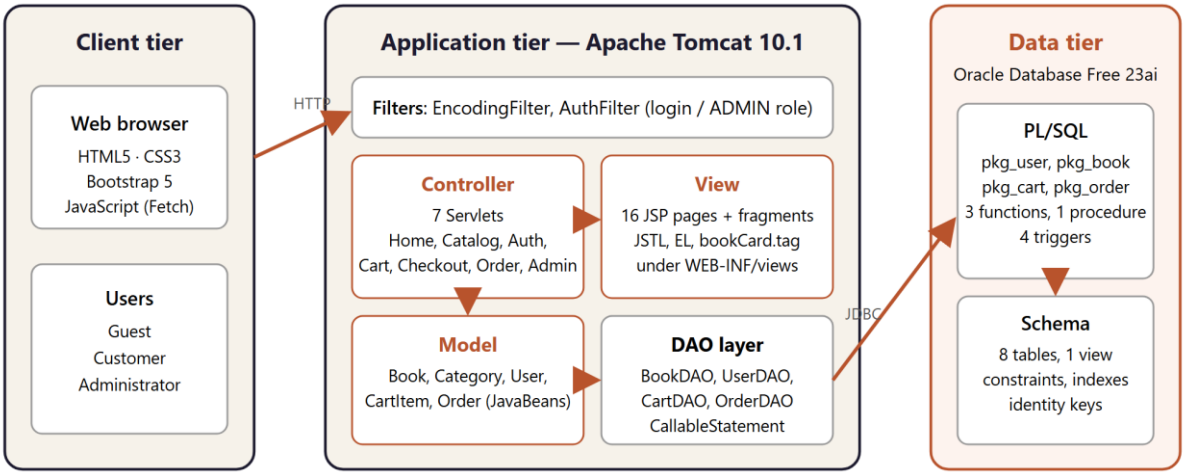


Figure 2.1: Three-tier architecture of ScriptWell with PL/SQL in the data tier

2.3 Data Flow and Transaction Workflow

The most important workflow is checkout. When a customer pays, a single call to `pkg_order.place_order` performs the following steps inside one transaction:

1. Validate the shipping address and payment method (private procedure `check_input`).
2. Raise the user-defined exception `e_empty_cart` if the cart has no rows.
3. Calculate the subtotal with `pkg_cart.cart_total` and the delivery charge with `fn_shipping_charge`.

4. Insert the order header and return the new `order_id` with `RETURNING ... INTO`.
5. Open an explicit cursor over the cart joined with BOOKS, locking each book row with `FOR UPDATE`.
6. For every line: check the stock, insert into ORDER_ITEMS and reduce the stock. Trigger `trg_books_stock_audit` logs each change.
7. Delete the cart rows and `COMMIT`. On any error, close the cursor and `ROLLBACK` the whole order.

2.4 Entity-Relationship Model and Data Dictionary

The schema contains six business tables and two audit tables. All primary keys are Oracle `IDENTITY` columns. Order numbers start at 1001. The view `V_BOOK_CATALOG` joins BOOKS with CATEGORIES and is used by every catalogue query.

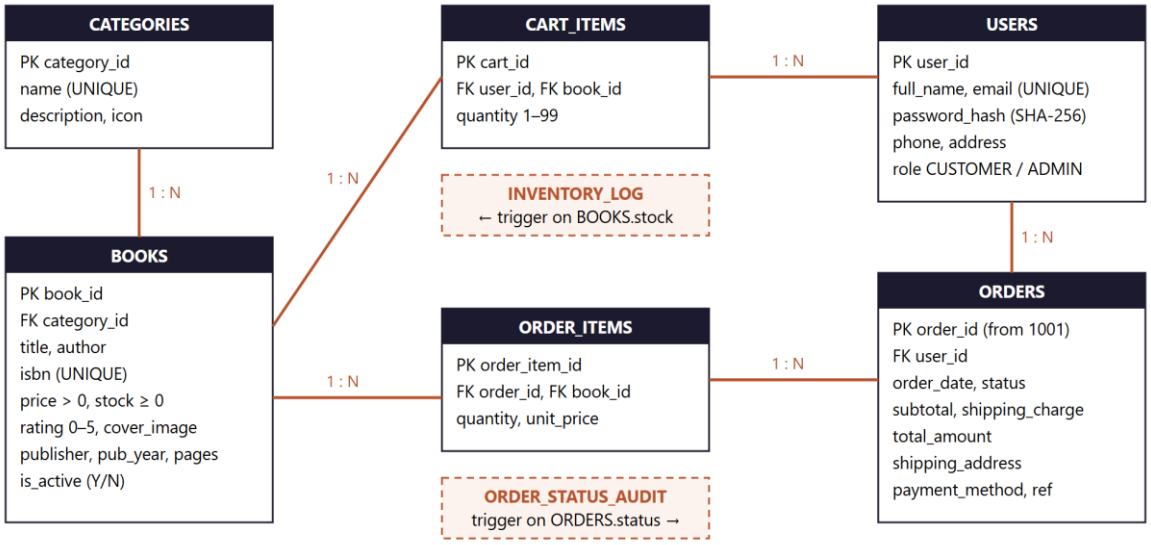


Figure 2.2: Entity-relationship diagram of the ScriptWell schema

Table 2.2: Data dictionary (main tables)

Table	Key columns	Constraints
CATEGORIES	category_id, name, description, icon	PK; name UNIQUE
BOOKS	book_id, title, author, isbn, category_id, price, stock, rating, is_active	PK; FK to CATEGORIES; isbn UNIQUE; CHECK price > 0, stock >= 0, rating 0-5, is_active IN (Y,N)
USERS	user_id, full_name, email, password_hash, phone, address, role	PK; email UNIQUE; CHECK role IN (CUSTOMER, ADMIN)
CART_ITEMS	cart_id, user_id, book_id, quantity	UNIQUE (user_id, book_id); CHECK quantity 1-99; ON DELETE CASCADE
ORDERS	order_id, user_id, subtotal, shipping_charge, total_amount, status, payment_method, payment_ref	PK from 1001; FK to USERS; CHECK status and payment_method
ORDER_ITEMS	order_item_id, order_id, book_id, quantity, unit_price	FKs to ORDERS (cascade) and BOOKS
INVENTORY_LOG	log_id, book_id, old_stock, new_stock, change_qty, reason	Written only by trigger
ORDER_STATUS_AUDIT	audit_id, order_id, old_status, new_status, changed_by	Written only by trigger

The design is in third normal form. Category names are stored once, and order lines are separate from order headers. The price at the moment of sale is copied deliberately into `ORDER_ITEMS.unit_price`, so

later price changes do not alter past orders. Indexes on `books(category_id)`, the function-based index `books(UPPER(title))`, `orders(user_id)` and `order_items(order_id)` speed up the most frequent queries.

2.5 Functional and Non-Functional Requirements

Functional requirements

- Register users with unique e-mail addresses and store only SHA-256 password hashes.
- Search books by keyword, category and price range, with sorting, through a single procedure.
- Maintain a persistent cart whose quantities never exceed the available stock.
- Place orders from the cart or with Buy Now, and generate a payment reference.
- Let customers cancel unshipped orders, returning the stock automatically.
- Let administrators run book CRUD, restock, update order status and view dashboard statistics.

Non-functional requirements

- **Consistency:** all-or-nothing checkout and no negative stock (CHECK constraint plus locking).
- **Security:** bind variables only, hashed passwords, and generic messages for unexpected errors.
- **Maintainability:** logic grouped into four packages, and scripts that can be re-run safely (00_install.sql).
- **Auditability:** every stock and order-status change is recorded automatically.

CHAPTER 3:

PL/SQL IMPLEMENTATION

3.1 Database Objects Overview

The PL/SQL code is organised into five script files that run in order from `00_install.sql`: schema, functions, packages, triggers and seed data. Table 3.1 summarises the program units.

Table 3.1: PL/SQL program units

Object	Type	Sub-programs / responsibility
pkg_user	Package	hash_password, register_user, login_user, get_all_users
pkg_book	Package	search_books, get_book, get_featured, get_new_arrivals, get_related, get_categories, add_book, update_book, delete_book, adjust_stock, get_low_stock, get_inventory_log, private validate_book
pkg_cart	Package	add_item (MERGE), update_item, remove_item, clear_cart, get_cart, cart_total, cart_count, private check_stock
pkg_order	Package	place_order, buy_now, get_user_orders, get_order, get_all_orders, update_status, cancel_order, sales_by_category, private restock_order and new_payment_ref
fn_shipping_charge	Function	Free delivery for subtotal >= Rs.499, otherwise Rs.49
fn_is_in_stock	Function	Returns Y/N for a requested quantity
fn_total_revenue	Function	Revenue of all non-cancelled orders
sp_get_dashboard_stats	Procedure	Six OUT parameters for the admin dashboard
4 triggers	Trigger	Data normalisation, validation, stock audit, order status audit

3.2 Packages and Stored Procedures

Each package has a **specification**, which is the public interface that Java calls, and a **body**, which holds the implementation. Helpers such as `validate_book` and `check_stock` exist only in the body, so they are private to the package. The specification of the cart package is shown below:

```
CREATE OR REPLACE PACKAGE pkg_cart AS
  PROCEDURE add_item    (p_user_id IN NUMBER, p_book_id IN NUMBER, p_qty IN NUMBER);
  PROCEDURE update_item (p_user_id IN NUMBER, p_book_id IN NUMBER, p_qty IN NUMBER);
  PROCEDURE remove_item (p_user_id IN NUMBER, p_book_id IN NUMBER);
  PROCEDURE clear_cart  (p_user_id IN NUMBER);
  PROCEDURE get_cart    (p_user_id IN NUMBER, p_result OUT SYS_REFCURSOR);
  FUNCTION  cart_total  (p_user_id IN NUMBER) RETURN NUMBER;
  FUNCTION  cart_count  (p_user_id IN NUMBER) RETURN NUMBER;
END pkg_cart;
```

Listing 3.1: Package specification of pkg_cart

Query procedures return their rows through a `SYS_REFCURSOR` OUT parameter. The catalogue search handles optional filters in one static statement. A NULL parameter disables its filter, and a CASE expression in the ORDER BY clause selects the sort order:

```
PROCEDURE search_books (p_keyword IN VARCHAR2, p_category_id IN NUMBER,
                        p_min_price IN NUMBER, p_max_price IN NUMBER,
                        p_sort IN VARCHAR2, p_result OUT SYS_REFCURSOR) IS
  v_kw VARCHAR2(210) := '%' || UPPER(TRIM(p_keyword)) || '%';
BEGIN
  OPEN p_result FOR
    SELECT * FROM v_book_catalog
     WHERE is_active = 'Y'
       AND (p_keyword IS NULL OR UPPER(title) LIKE v_kw OR UPPER(author) LIKE v_kw
            OR UPPER(isbn) LIKE v_kw OR UPPER(category_name) LIKE v_kw)
       AND (p_category_id IS NULL OR category_id = p_category_id)
       AND (p_min_price IS NULL OR price >= p_min_price)
       AND (p_max_price IS NULL OR price <= p_max_price)
     ORDER BY CASE WHEN p_sort = 'price_asc'  THEN price  END ASC,
              CASE WHEN p_sort = 'price_desc' THEN price  END DESC,
              CASE WHEN p_sort = 'rating'     THEN rating END DESC,
              title;
END search_books;
```

Listing 3.2: Search procedure returning a REF CURSOR

The cart uses a `MERGE` statement, so adding a book that is already in the cart increases its quantity instead of creating a duplicate row. Before the MERGE, the private procedure `check_stock` makes sure the combined quantity is available.

3.3 Stored Functions

Stand-alone functions hold small, reusable rules that can be called from SQL, from other PL/SQL code and from Java. `fn_is_in_stock` also shows how a predefined exception is handled:

```
CREATE OR REPLACE FUNCTION fn_shipping_charge (p_subtotal IN NUMBER) RETURN NUMBER IS
BEGIN
  IF p_subtotal IS NULL OR p_subtotal = 0 THEN RETURN 0;
  ELSIF p_subtotal >= 499 THEN RETURN 0;
  ELSE RETURN 49;
  END IF;
END fn_shipping_charge;

CREATE OR REPLACE FUNCTION fn_is_in_stock (p_book_id IN NUMBER, p_qty IN NUMBER)
RETURN VARCHAR2 IS
  v_stock books.stock%TYPE;
BEGIN
  SELECT stock INTO v_stock FROM books WHERE book_id = p_book_id AND is_active = 'Y';
  RETURN CASE WHEN v_stock >= p_qty THEN 'Y' ELSE 'N' END;
EXCEPTION
  WHEN NO_DATA_FOUND THEN RETURN 'N';
END fn_is_in_stock;
```

Listing 3.3: Stored functions

The stand-alone procedure `sp_get_dashboard_stats` returns six values (active books, customers, orders, revenue, low-stock books and pending orders) through OUT parameters in one round trip. It calls `fn_total_revenue` internally, so one stored program reuses another.

3.4 Cursors and Transaction Control

`pkg_order.place_order` uses an **explicit cursor** with `FOR UPDATE OF b.stock`. Opening the cursor locks every book in the customer's cart until the transaction ends. If two customers try to buy the last copy, the second one waits, then sees the reduced stock and receives error -20003. If anything fails, the handler closes the cursor and rolls back.

```
CURSOR c_cart IS
  SELECT c.book_id, c.quantity, b.price, b.stock, b.title
    FROM cart_items c JOIN books b ON b.book_id = c.book_id
   WHERE c.user_id = p_user_id
     FOR UPDATE OF b.stock;
r_item c_cart%ROWTYPE;
...
OPEN c_cart;
LOOP
  FETCH c_cart INTO r_item;
  EXIT WHEN c_cart%NOTFOUND;
  IF r_item.stock < r_item.quantity THEN
    RAISE_APPLICATION_ERROR(-20003, 'Only ' || r_item.stock || ' copies of "'
                            || r_item.title || '" left in stock');
  END IF;
  INSERT INTO order_items (order_id, book_id, quantity, unit_price)
  VALUES (p_order_id, r_item.book_id, r_item.quantity, r_item.price);
  UPDATE books SET stock = stock - r_item.quantity WHERE book_id = r_item.book_id;
END LOOP;
CLOSE c_cart;
DELETE FROM cart_items WHERE user_id = p_user_id;
COMMIT;
EXCEPTION
  WHEN e_empty_cart THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20004, 'Your cart is empty');
  WHEN OTHERS THEN
    IF c_cart%ISOPEN THEN CLOSE c_cart; END IF;
    ROLLBACK;
    RAISE;
```

Listing 3.4: Explicit cursor with row locking in place_order

A **cursor FOR loop** is used when an order is cancelled. The private procedure `restock_order` loops over the order lines and returns each quantity to stock. Oracle opens, fetches from and closes the cursor automatically:

```
PROCEDURE restock_order (p_order_id IN NUMBER) IS
  CURSOR c_lines IS SELECT book_id, quantity FROM order_items WHERE order_id = p_order_id;
BEGIN
  FOR r IN c_lines LOOP
    UPDATE books SET stock = stock + r.quantity WHERE book_id = r.book_id;
  END LOOP;
END restock_order;
```

Listing 3.5: Cursor FOR loop

Implicit cursor attributes are also used. For example, `update_book` checks `SQL%ROWCOUNT = 0` to detect a book that does not exist, and raises -20005.

3.5 Triggers

Table 3.2: Row-level triggers

Trigger	Timing and event	Action
trg_users_normalize	BEFORE INSERT OR UPDATE OF email	Lower-cases and trims the e-mail; rejects values without '@'
trg_books_before_save	BEFORE INSERT OR UPDATE	Trims title and author, upper-cases ISBN, rejects a future publication year
trg_books_stock_audit	AFTER INSERT OR UPDATE OF stock	Writes old stock, new stock, change and reason to INVENTORY_LOG
trg_orders_status_audit	AFTER INSERT OR UPDATE OF status	Writes the status history and session user to ORDER_STATUS_AUDIT

```
CREATE OR REPLACE TRIGGER trg_books_stock_audit
AFTER INSERT OR UPDATE OF stock ON books
FOR EACH ROW
DECLARE
  v_reason inventory_log.reason%TYPE;
BEGIN
  IF INSERTING THEN                v_reason := 'INITIAL STOCK';
  ELSIF :NEW.stock < :OLD.stock THEN v_reason := 'SALE / ADJUSTMENT';
  ELSE                              v_reason := 'RESTOCK / RETURN';
  END IF;
  IF INSERTING OR :NEW.stock <> :OLD.stock THEN
    INSERT INTO inventory_log (book_id, old_stock, new_stock, change_qty, reason)
    VALUES (:NEW.book_id, :OLD.stock, :NEW.stock, :NEW.stock - NVL(:OLD.stock, 0), v_reason);
  END IF;
END;
```

Listing 3.6: Stock audit trigger using :NEW, :OLD and INSERTING

3.6 Exception Handling

Three kinds of exceptions are used. **Predefined** exceptions (`NO_DATA_FOUND`, `DUP_VAL_ON_INDEX`) are caught and turned into clear messages. The **user-defined** exception `e_empty_cart` is declared in the `pkg_order` specification and raised with `RAISE`. Business-rule violations call **RAISE_APPLICATION_ERROR** with codes reserved for this application:

Table 3.3: Application error codes

Code	Meaning	Raised by
-20001	E-mail already registered	register_user (also maps DUP_VAL_ON_INDEX)
-20002	Invalid e-mail or password	login_user (maps NO_DATA_FOUND)
-20003	Insufficient stock	check_stock, place_order, buy_now
-20004	Cart is empty	place_order (from e_empty_cart)
-20005 / -20008	Book / order not found	book and order procedures
-20006	Invalid quantity or negative stock	check_stock, adjust_stock
-20007 / -20009	Order cannot be cancelled / invalid status change	cancel_order, update_status
-20010	Validation failed	validation helpers and triggers

3.7 Calling PL/SQL from Java (JDBC CallableStatement)

The web tier calls the packages through JDBC escape syntax. IN parameters are bound with setter methods, OUT parameters are registered before execution, and a REF CURSOR is read as a normal `ResultSet`. When Oracle raises an error between -20000 and -20999, Java shows its message to the user. Any other error is logged and replaced with a generic message.

```
try (Connection con = DBConnection.getConnection();
     CallableStatement cs = con.prepareCall("{call pkg_order.place_order(?,?,?,?,?)}")) {
    cs.setInt(1, userId);
    cs.setString(2, address);
    cs.setString(3, payment);                    // CARD, UPI or COD
    cs.registerOutParameter(4, Types.NUMERIC);   // new order id
    cs.registerOutParameter(5, Types.VARCHAR);   // payment reference
    cs.execute();
    return new Object[] { cs.getInt(4), cs.getString(5) };
}
```

Listing 3.7: Calling place_order from OrderDAO.java

CHAPTER 4:

TESTING, RESULTS AND CONCLUSION

4.1 Testing Methodology

The database was tested at two levels. **Unit testing** called each procedure and function directly in SQL*Plus with valid, invalid and boundary values, then checked the returned values, the rows affected, the audit rows written by triggers and the error codes raised. **Integration testing** ran the same operations through the web application, confirming that the Java DAOs pass parameters correctly and show the PL/SQL messages to the user. The install script `00_install.sql` lists any compilation errors from `USER_ERRORS`, and none were reported.

4.2 Test Cases and Validation

Table 4.1: PL/SQL test cases

No.	Test case	Expected result	Status
1	register_user with an existing e-mail	ORA-20001 'An account with this email already exists'	Pass
2	register_user with a 3-character password	ORA-20010 validation error	Pass
3	login_user with a wrong password	ORA-20002 'Invalid email or password'	Pass
4	search_books('java', NULL, NULL, NULL, NULL)	2 rows returned	Pass
5	search_books with category 5, sort price_desc	6 rows, highest price first	Pass
6	add_item: 5 copies of a book with stock 1	ORA-20003 'Only 1 copy ... available'	Pass
7	place_order with an empty cart	ORA-20004 'Your cart is empty'	Pass
8	place_order with a valid cart	Order created, stock reduced, cart emptied, log rows added	Pass
9	cancel_order on a PLACED order	Status CANCELLED, stock restored by cursor loop	Pass
10	update_status of a CANCELLED order to PLACED	ORA-20009 'A cancelled order cannot be reopened'	Pass
11	add_book with price -5 / year 2099	ORA-20010 from validate_book / trigger	Pass
12	delete_book on a book that has been sold	Book deactivated (is_active = 'N'), not deleted	Pass
13	fn_shipping_charge(300) / fn_shipping_charge(600)	49 / 0	Pass

4.3 Results and Analysis

The seed script places four orders through the packages. The queries below show that the procedures, functions and triggers worked together correctly. Order 1003 (subtotal Rs.349) was charged Rs.49 delivery, and every sale and status change was audited automatically.

```
SQL> SELECT order_id, subtotal, shipping_charge, total_amount, status, payment_method FROM orders;
  ORDER_ID   SUBTOTAL SHIPPING_CHARGE TOTAL_AMOUNT STATUS     PAYMENT_METHOD
      1001       1598               0         1598 DELIVERED  CARD
      1002       2096               0         2096 SHIPPED    UPI
      1003        349              49          398 PLACED     COD
      1004        749               0          749 PLACED     CARD

SQL> SELECT book_id, old_stock, new_stock, change_qty, reason FROM inventory_log
     WHERE reason <> 'INITIAL STOCK';
   BOOK_ID  OLD_STOCK  NEW_STOCK CHANGE_QTY REASON
        31         65         64         -1 SALE / ADJUSTMENT
        11         45         43         -2 SALE / ADJUSTMENT      ... (8 rows selected)

SQL> SELECT name, type FROM user_errors;
no rows selected
```

*Listing 4.1: Verification queries in SQL*Plus*

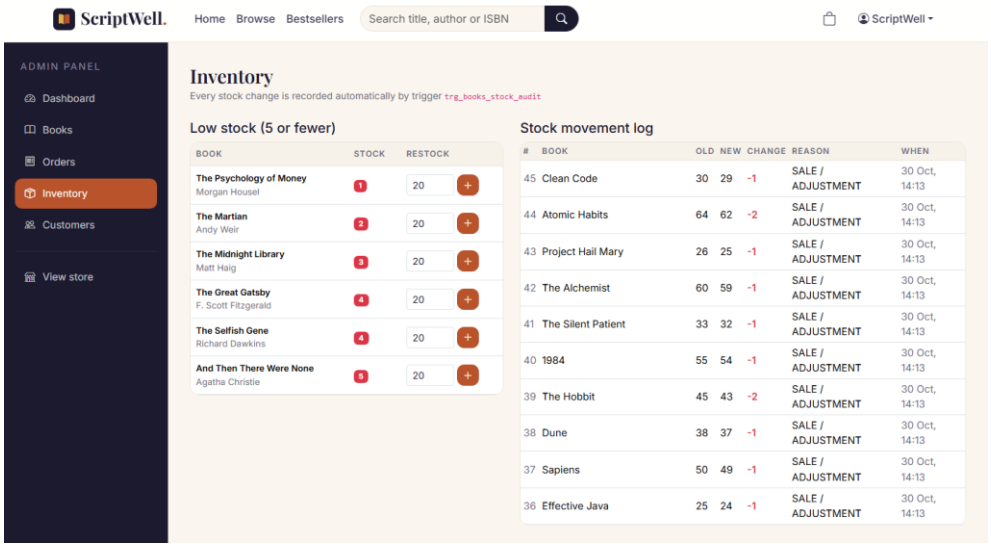


Figure 4.1: Inventory page showing the stock movement log written by trg_books_stock_audit

4.4 Limitations and Future Scope

- Payment is simulated. A real gateway would need a confirmation table and webhook handling.
- Only one warehouse is modelled. A STOCK_LOCATIONS table could support multiple warehouses.
- Coupons and discounts could be added as a new `pkg_promo` package with validity rules.
- Reports such as monthly sales could use analytic functions and materialised views.
- Bulk operations could use `BULK COLLECT` and `FORALL` for very large catalogues.

4.5 Conclusion

The ScriptWell database meets all of its objectives. The schema is normalised and protected by constraints. Business operations are grouped into four packages with clear public interfaces. Stored functions and a stored procedure provide reusable calculations. Explicit cursors with row locking make checkout atomic and safe when many customers order at once, triggers keep complete audit trails without any application code, and structured exception handling gives clear, consistent messages. Because the rules live in PL/SQL, the Java application is simpler, and the data stays correct whichever client uses the database.

4.6 Pushing the Project to GitHub

All SQL scripts, the Java source code and this report are under version control with Git and are published to GitHub at **https://github.com/lovishv1718/scriptwell**. The commands used were:

```
git init -b main
git add .
git commit -m "ScriptWell online book store (Servlets, JSP, JDBC, Oracle PL/SQL)"
git remote add origin https://github.com/lovishv1718/scriptwell.git
git push -u origin main
```

REFERENCES-

[1] Oracle Corporation, *Oracle Database PL/SQL Language Reference, 23ai*, docs.oracle.com.

[2] Oracle Corporation, *Oracle Database SQL Language Reference, 23ai*, docs.oracle.com.

[3] Oracle Corporation, *Oracle Database JDBC Developer's Guide, 23ai*, docs.oracle.com.

[4] S. Feuerstein and B. Pribyl, *Oracle PL/SQL Programming*, 6th ed., O'Reilly Media, 2014.

[5] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.

[6] ScriptWell source code repository, https://github.com/lovishv1718/scriptwell.